

Data Modeling Documentation

A practical guide to designing reliable databases from first principles

This document explains how to move from business understanding to entities, relationships, keys, rules, history, and performance decisions when modeling a database.

1. What data modeling is

Data modeling is the process of designing how information is structured, connected, stored, and governed in a system. A good model does not just describe tables. It represents the business clearly enough that the data stays consistent and the system can answer real questions reliably.

- What data do we need?
- What are the important business objects?
- How do these objects relate to one another?
- What rules must always be true?
- How should the data change over time?

2. The right starting point: understand the business

The first mistake many beginners make is starting with table names. The better starting point is the business process. Before creating anything, understand what the system does, who uses it, and what questions it must answer.

Question	Why it matters
What process are we modeling?	Defines the true scope of the database.
Who creates or updates the data?	Reveals transactional flows and ownership.
What reports or screens are needed?	Helps design for real usage, not just storage.
What rules can never be broken?	Becomes constraints, validations, and checks.
Do values change over time?	Determines whether audit or history tables are needed.

3. Core building blocks of a model

Every solid data model is built from a few core ideas:

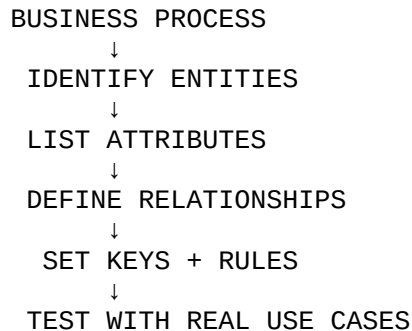
Entities: The main things the business cares about, such as Customer, Product, Order, Invoice, or Employee.

Attributes: The details you need to store about each entity, such as customer name, order date, or product price.

Relationships: How one entity connects to another, such as one customer placing many orders.

Rules: Conditions that must hold true, such as email being unique or quantity being greater than zero.

Simple modeling flow



4. Keys and relationships

Keys and relationships give the model structure and integrity. Without them, the database becomes a set of disconnected spreadsheets.

Primary key: Uniquely identifies a row, for example `customer_id` or `order_id`.

Foreign key: Links one table to another, for example `orders.customer_id` referencing `customers.customer_id`.

One-to-many: One parent row can relate to many child rows. Example: one customer to many orders.

Many-to-many: Requires a junction table. Example: students and courses connect through enrollments.

Optional vs mandatory: Some relationships must exist and some can be blank. This affects whether foreign keys allow NULL.

Customers	Orders	Order Items
customer_id (PK) name email	order_id (PK) customer_id (FK) order_date	order_item_id (PK) order_id (FK) product_id (FK) quantity
1 customer	places many	contains many
Products	product_id (PK) product_name price	Each order item refers to one product

5. Normalization and data quality

Normalization means organizing the data so that one fact is stored once in the right place. This reduces duplication, inconsistency, and update problems.

- Avoid storing customer details repeatedly inside every order row.
- Use separate tables for core entities and connect them with keys.
- Keep repeating groups, such as multiple products in one order, in a separate child table like `order_items`.
- Normalize first for correctness. Denormalize only when performance or analytics clearly require it.

Important: a clean model is not only about fewer tables. It is about placing each business fact where it logically belongs.

6. Constraints and business rules

A model should reflect business truth. These truths are usually enforced with constraints in the database and validation in the application layer.

Rule type	Example
Unique	A user email must not appear twice.
Not null	An order must belong to a customer.
Check	Quantity must be greater than zero.
Reference integrity	A payment cannot exist for an order that does not exist.
Controlled values	Status should come from an approved set such as pending, paid, shipped, cancelled.

7. History, audit, and change over time

Many weak data models only capture the current state. Stronger models also think about time. Ask what should happen when prices, addresses, statuses, or assignments change.

- Use `created_at` and `updated_at` when you need a basic audit trail.
- Use `deleted_at` for soft delete when records should not vanish completely.
- Use `valid_from` and `valid_to` when you need to keep old versions of data.
- Make a clear decision between storing only the latest value and storing full history.

8. Performance and real-world use

A model should not only be correct. It should also support the way the system will be used. Think early about common reads, writes, joins, and reports.

Indexing: Index keys and frequently filtered columns such as `customer_id`, `order_date`, or `email`, but do not index everything blindly.

Data types: Choose types carefully. Use `DECIMAL` for money, `TIMESTAMP` for event times, and proper numeric or boolean fields instead of storing everything as text.

Transaction vs analytics: Operational databases usually favor normalized designs, while reporting systems often use fact and dimension tables.

Scale: The design for a system with 1,000 rows may not suit one with 100 million rows.

9. Step-by-step modeling process

1. Understand the business process and scope.
2. List the core entities.
3. List the important attributes for each entity.
4. Define relationships and cardinality.
5. Choose primary keys and foreign keys.
6. Add business rules and constraints.
7. Normalize the structure.
8. Plan how changes and history should be handled.
9. Review common queries and indexing needs.
10. Test the model with real user scenarios.
11. Document the final structure with an ERD and table definitions.

10. What a proper data modeling document should contain

- Purpose and scope of the model
- Business context and process description
- Entity list with definitions
- Attribute list with meanings
- Primary keys, foreign keys, and relationship notes
- Business rules and data quality constraints
- History or audit behavior
- Performance notes such as indexes or partitioning
- Sample scenarios showing how the model behaves

Best mindset: do not ask only “What tables should I create?” Ask “What business am I representing, what must stay true, and how will this data be used over time?”